ARCHITECTURE FIRST

GPT-SaaS

AI 对话知识库编译系统

把混杂的多条对话，编译成可追溯、可更新的学习知识文件

01	问答档案	知道过去问过什么
02	跨对话归纳	打破原始聊天边界
03	知识文件树	生成可学习的稳定结构
04	增量演化	新对话只更新相关内容

重新审核版 v2.0 | 2026-09-12 | 双产物与知识编译蓝图

目录

1. 重新审核后的产品定义 4
2. 典型场景：一个“学习”项目 4
3. 两套核心产物 4
3.1 产物 A：原始问答档案 4
3.2 产物 B：知识文件树 4
4. 知识文件的标准结构 5
5. 必须区分的四种内容 5
6. 产品信息架构 6
7. 核心领域对象 6
8. 数据分层 6
L0 原始证据 6
L1 规范化会话 7
L2 知识片段与语义对象 7
L3 知识文件编译结果 7
L4 发布与学习视图 7
9. 端到端“编译”流水线 7
10. 跨对话归纳方法 7
10.1 先切片，再归类 7
10.2 层级归类而非单标签 8
10.3 框架生成与稳定性 8
11. 知识文件合成策略 8
12. AI 扩展知识的正确位置 8
13. 增量更新与知识演化 8
14. 问题树、任务与决策的定位 9
15. 时间线应该展示什么 9
16. 推荐系统架构 9
16.1 初期采用模块化单体 9
16.2 模块边界 10
17. 数据接入顺序 10
18. AI 调用编排与 API 10
18.1 分阶段调用 10
18.2 OpenAI 调用建议 10
18.3 供应商适配器 11
19. 自有后端 API 设计 11

20. 数据库与检索策略 11
21. 隐私、安全与可信性 12
22. UI 实现顺序 12
23. 实现版本路线 12
v0.1 原始项目与问答档案（第 1–2 周） . . . 12
v0.2 知识片段池（第 3–4 周） . . . 12
v0.3 知识框架与文件编译（第 5–6 周） . . . 12
v0.4 AI 补充与人工审核（第 7–8 周） . . . 12
v0.5 增量编译与时间线（第 9–10 周） . . . 12
v0.6 可用 UI 与导出（第 11–12 周） . . . 12
v0.8 多供应商与多项目 . . . 13
v1.0 稳定知识编译平台 . . . 13
24. 测试与评测 13
25. 成本与性能 13
26. 建议仓库结构 13
27. 主要风险与对策 14
28. 最终实施原则 14
29. 参考资料（核对日期：2026-09-12） 14

1. 重新审核后的产品定义

本产品的核心不是“给聊天记录分类”，也不只是“展示问题树”。它是一套 AI 对话知识库编译系统：输入是一个 ChatGPT 项目中杂乱、重复、跨主题的多条对话；输出是两套互相连接的结构化产物。

第一套产物是“原始问答档案”，回答用户过去问过什么、得到过什么回答、问题是否解决。第二套产物是“跨对话知识文件树”，把不同对话里的相关内容打散、去重、重组为适合学习和长期复习的文件结构。

最准确的处理链是：

ChatGPT 项目 -> 全部对话 -> 原始问答索引 -> 知识片段池 -> 跨对话归并 -> 知识文件树 -> AI 扩展知识 -> 增量更新

产品价值发生在“跨对话归并”这一层，而不是发生在原始聊天标题上。

2. 典型场景：一个“学习”项目

用户在 ChatGPT 中创建“学习”项目，里面可能有以下对话：

- 学习方法
- 学习规划
- 高等数学
- 数据结构
- 英语学习

真实内容不会服从标题边界。“学习方法”中可能讨论高数错题；“高等数学”中可能讨论时间安排；“学习规划”中可能讨论间隔复习；“英语学习”中也可能出现通用的主动回忆方法。

因此，“会话”只能作为来源容器，不能作为最终知识分类。系统必须读取整个项目的全部授权对话，根据语义把内容重新组织。例如，来自四条对话的主动回忆、间隔复习和错题回顾内容，最终都可以进入 02 通用学习方法/间隔复习.md。

3. 两套核心产物

3.1 产物 A：原始问答档案

这一层忠实保留历史，主要解决“我是否问过”和“当时如何回答”。建议结构：

```
原始问答档案/
  按时间/
  按原始会话/
  按问题类型/
  重复问题/
  尚未解决/
```

每个问题至少保存：原始文本、直接回答、对话与消息定位、提问时间、问题标签、是否解决、后续修订、涉及的知识文件。

3.2 产物 B：知识文件树

这一层服务于学习、复习和继续研究。它不复制原始会话结构，而是生成新的主题框架：

```
学习知识库/
  01 学习目标与规划/
    长期目标.md
    学期规划.md
```

```
    每周与每日计划.md
  02 通用学习方法/
    主动回忆.md
    间隔复习.md
    错题整理.md
    知识输出.md
  03 时间与精力管理/
    时间分配.md
    专注管理.md
    疲劳与休息.md
  04 专业课学习/
    高等数学/
    数据结构/
  05 待验证与冲突/
    尚无明确答案.md
    相互冲突的建议.md
```

两套产物不是二选一。问答档案保护原貌，知识文件树提供新的认知结构；每个知识段落都必须可以跳回它使用的原始问题和回答。

4. 知识文件的标准结构

每个 AI 归纳文件不应只有一段摘要。推荐固定为八个区块：

```
# 间隔复习

## 1. 核心结论
一页内可理解的最终归纳。

## 2. 从历史对话中归纳
只使用项目内已有对话，并保留逐段来源。

## 3. 方法框架
对历史内容去重、排序和结构化后的知识框架。

## 4. 适用条件与例外
说明方法何时有效、何时不适用。

## 5. AI 补充知识
明确标识为外部补充，不冒充历史对话结论。

## 6. 可执行清单
把知识转换为行动步骤与检查点。

## 7. 冲突与待解决问题
保留不同回答的矛盾、低置信度判断和缺口。

## 8. 来源
原始会话、消息、文件和分析版本。
```

文件还应包含机器可读 front matter：`project_id`、`knowledge_document_id`、`source_message_ids`、`framework_version`、`compiled_at`、`compilation_run_id`。这样文件既适合人阅读，也能被系统安全更新。

5. 必须区分的四种内容

如果所有文字混在一起，用户会无法判断知识来自哪里。系统必须在数据和界面上区分：

内容类型	含义	展示要求
Source-derived	从用户历史对话直接归纳	必须有消息级来源
AI-synthesized	AI 对多条历史内容进行重组	必须列出所依据的片段
AI-supplemented	模型根据通用知识新增	显示“AI 补充”，不可伪装成历史结论
User-approved	用户确认后的正式知识	保存确认人、时间与版本

任何缺少来源的历史归纳只能标记为 proposed。AI 补充可以没有项目内来源，但必须记录模型、提示版本、生成时间，并允许用户关闭这一功能。

6. 产品信息架构

建议采用以下层级：

Workspace -> ImportedProject -> Conversation -> Message

这是来源侧结构。知识侧结构独立存在：

KnowledgeBase -> Folder -> KnowledgeDocument -> Section -> Statement

两侧通过 KnowledgeFragment 与 Citation 连接。问题、决策和任务是语义对象，不强迫它们成为文件树的唯一主轴：

Question -> AnswerCandidate -> KnowledgeFragment -> KnowledgeSection

Decision/Task/Problem -> related_to -> KnowledgeDocument

由此可以做到：一个对话片段进入多个知识文件；一个知识文件综合多个对话；同一段内容在不同上下文中承担不同作用，但只保存一份原始证据。

7. 核心领域对象

对象	作用	关键字段
ImportedProject	一次导入的 ChatGPT/AI 项目	provider、external_id、title、source_hash
Conversation	原始对话容器	title、started_at、branch_structure
Message	不可变原始消息	role、content、created_at、parent_id
Question	从消息中识别的问题	normalized_text、status、asked_at
AnswerCandidate	与问题相关的回答	message_id、directness、quality
KnowledgeFragment	可被重新编排的最小语义片段	text、type、source_locator、embedding
Concept	跨对话对齐的概念	canonical_name、aliases、scope
KnowledgeDocument	生成的知识文件	path、title、status、framework_version
KnowledgeSection	文件内固定区块	section_type、ordinal、content
Statement	文件中的原子结论	provenance_type、confidence、validity
Citation	Statement 到原始消息的引用	message_id、start、end、quote_hash
Supplement	AI 新增的知识	model、prompt_version、review_state
CompilationRun	一次知识库编译作业	input_hash、pipeline_version、cost
Revision	文件或结论的版本变化	before、after、reason、actor

关系包括 DERIVED_FROM、ANSWERS、ABOUT、MERGED_WITH、SUPPORTS、CONTRADICTS、REFINES、SUPERSEDES、PLACED_IN、UPDATED_BY。

8. 数据分层

L0 原始证据

只追加、不覆盖。保存导出包、原始 JSON/HTML、消息顺序、分支、附件、哈希和导入授权。L0 是最终追溯依据。

L1 规范化会话

把供应商格式转换为统一 Conversation、Message、Attachment、SourceLocator 和 Canonical Event。这里只做确定性转换。

L2 知识片段与语义对象

从消息中提取 Question、AnswerCandidate、KnowledgeFragment、Concept、Task、Decision 和关系。每个片段保持消息级 locator。

L3 知识文件编译结果

根据目标框架生成 Folder、KnowledgeDocument、KnowledgeSection 和 Statement。结果可重编译、可比较、可人工修订。

L4 发布与学习视图

为问答检索、知识文件树、复习卡片、时间线和导出建立发布版本。发布层不反向覆盖 L0-L3。

9. 端到端“编译”流水线

1. `ingest`：上传项目导出包，计算 SHA-256，登记来源和权限。
2. `normalize`：解析全部对话、分支、消息、附件和时间。
3. `question-index`：识别显式/隐式问题，连接对应回答，建立问答档案。
4. `fragment`：按论点和用途切成知识片段，不按固定 token 粗暴截断。
5. `tag`：识别主题、范围、方法类型、学科、适用条件和敏感性。
6. `retrieve-candidates`：关键词与向量混合召回可能属于同一概念的片段。
7. `cluster`：形成候选主题簇，允许一个片段进入多个候选簇。
8. `resolve`：去重、概念对齐、拆分过宽主题、识别冲突。
9. `framework`：为整个项目提出知识文件树，而不是逐聊天生成摘要。
10. `compose`：按文件模板写入来源归纳、方法框架、例外和行动清单。
11. `supplement`：可选生成 AI 扩展知识，写入独立区块。
12. `validate`：检查引用覆盖、事实支持、重复段落、文件层级与循环引用。
13. `review`：用户审核低置信度归类、冲突和重要补充。
14. `publish`：生成新的知识库版本，并更新反向引用。

每一步都是独立、幂等、可重试的 Job。编译指纹建议为：

```
project_source_hash + pipeline_version + framework_version + prompt_version + model_snapshot
```

10. 跨对话归纳方法

10.1 先切片，再归类

不要让模型直接对每个会话写总结，否则会固化错误的会话边界。先提取带来源的 KnowledgeFragment，再在全项目范围聚类。

片段类型至少包括：问题、直接答案、方法、原则、例子、反例、限制条件、行动步骤、待验证主张、个人偏好。切片时保存前后消息窗口，避免一句话脱离上下文。

10.2 层级归类而非单标签

同一片段可以同时属于：

- 通用学习方法 / 主动回忆
- 专业课学习 / 高等数学 / 错题复习
- 时间管理 / 每周复盘

但发布时要避免重复大段正文。推荐一个主归属文件，其他文件使用引用卡片或链接。

10.3 框架生成与稳定性

第一次导入后生成 Framework v1。框架节点需要稳定 ID，重命名文件不能改变知识身份。后续新对话优先映射到已有文件；只有确有新主题、旧文件过宽或冲突显著时，才提出新增、拆分或合并。

11. 知识文件合成策略

合成采用 Map -> Reduce -> Critique 三阶段：

1. Map：分别对每个主题簇提取结论、方法、条件、例外和来源。
2. Reduce：去重并形成结构，禁止生成没有对应 fragment 的“历史结论”。
3. Critique：逐条检查是否被来源支持、是否把课程特定方法错误提升为通用方法、是否遗漏反例。

每条 Statement 保存 provenance_type。当多个回答冲突时，不要强行生成单一答案，应写入“冲突与待解决问题”，并显示各自适用条件或等待用户选择。

12. AI 扩展知识的正确位置

AI 补充很有价值，因为它可以把用户零散讨论提升为更完整的学习框架。但它必须是第二阶段，而不能污染第一阶段的历史归纳。

推荐顺序：

1. 仅根据项目内容生成“来源归纳版”。
2. 冻结并检查引用。
3. 把归纳版作为输入，请 AI 指出缺少的概念、可能的理论联系和实践建议。
4. 所有新增内容写入 AI 补充知识，默认状态为 proposed。
5. 用户接受后可以升级为正式知识，但来源类型仍保留 AI-supplemented。

可以为 AI 补充设置模式：关闭、保守补充、完整扩展。学习类内容还应区分“通用建议”和“专业学科事实”。

13. 增量更新与知识演化

新增对话后不应全量重写整个知识库。处理流程：

1. 只解析新增或变化的消息。

2. 提取新片段并召回最相关的已有知识文件。
3. 生成 No change / Append / Revise / Split / Merge / Conflict 六类提案。
4. 展示段落级 diff 和新来源。
5. 用户确认后产生新 Revision。
6. 重新计算受影响文件的摘要、交叉链接和复习材料。

必须保留旧版本，支持查看“哪条新对话改变了哪条知识”。如果一个历史结论后来被修正，使用 SUPERSEDES 关系，不直接删除旧结论。

14. 问题树、任务与决策的定位

复杂问题拆解仍然重要，但它是知识系统的一项能力，而不是整个产品的唯一中心。

问题树适合研究任务：一个大问题被拆为若干可验证子问题。任务与决策适合把知识转化为行动。它们都应连接知识文件：问题的答案可以沉淀进文件；文件中的待解决项可以重新转成问题；行动清单可以生成任务；用户采纳某个方案时生成 Decision。

拆解仍采用候选树 + Critique，并在“单一负责人、输入输出明确、有验收标准”时停止继续拆分。

15. 时间线应该展示什么

时间线不是完整消息流水账，而是知识发生变化的记录。建议五条泳道：

- 问答：首次提出、再次追问、得到新答案。
- 概念：新概念出现、概念合并或范围调整。
- 文件：知识文件新增、拆分、重命名、更新。
- 结论：Statement 被支持、反驳、修订或取代。
- 行动：任务、决定与实践结果。

示例：9 月 1 日提出“固定复习间隔”；9 月 5 日在高数对话中发现难度不同；9 月 8 日把结论修订为“按回忆表现动态调整”。每个事件必须跳回来源和 Revision。

16. 推荐系统架构



16.1 初期采用模块化单体

- Web：Next.js/React，后期实现三栏知识工作台。
- API：TypeScript + Fastify，承担权限、事务和 REST/SSE。
- Worker：Node.js 队列消费者；复杂文档解析可用独立 Python worker。
- 数据库：PostgreSQL + pgvector，先同时承担关系数据、全文和向量检索。
- 对象存储：S3 兼容服务，保存原始导出与附件。
- 队列：Redis + BullMQ；规模证明后再引入 Temporal。

16.2 模块边界

identity、ingestion、normalization、question-index、fragmentation、knowledge-compiler、llm-gateway、search、timeline、publishing、audit。

连接器只能写 L0/L1；AI worker 只能提交推导结果；发布器只能从审核后的编译结果生成文件，禁止模型直接覆盖原始记录。

17. 数据接入顺序

顺序	来源	MVP 方法	目标
1	单个 ChatGPT 项目导出	用户上传官方导出 JSON/HTML	跑通多对话编译闭环
2	Markdown/JSON	文件或目录导入	接纳现有学习笔记
3	PDF 与附件	上传、解析、页码定位	保留学习材料依据
4	自家 AI 会话	调用模型时同步记录	获得完整请求、响应和 trace
5	Claude/Gemini 等导出	连接器适配	验证供应商无关模型
6	Git 或外部知识库	授权后增量同步	扩展到代码与团队项目

不要把“调用模型 API”与“读取消费端聊天历史”混为一谈。首版应采用用户主动导入；第三方实时连接只有在官方权限、稳定性与条款明确后再做。

18. AI 调用编排与 API

18.1 分阶段调用

至少拆为 question-extract、fragment-extract、topic-route、cluster-label、framework-propose、document-compose、citation-critic、supplement 和 change-propose。每类任务使用独立提示、JSON Schema、模型配置和评测集。

18.2 OpenAI 调用建议

新实现以 Responses API 为主。结构化抽取使用 JSON Schema/Structured Outputs；耗时编译使用应用队列或 background；大量历史回填使用 Batch API。Files/Vector Store 可以帮助文件检索，但自有 L0/L1 仍是权威数据。

```
const result = await openai.responses.create({
  model: process.env.ANALYSIS_MODEL!,
  input: [{ role: "user", content: fragmentWindow }],
  instructions: FRAGMENT_EXTRACTION_PROMPT_V4,
  text: {
    format: {
      type: "json_schema",
```

```
        name: "knowledge_fragment_extraction",
        strict: true,
        schema: knowledgeFragmentSchema
      }
    },
    store: false,
    metadata: { compilation_run_id: runId, message_id: messageId }
  });
```

模型名称通过配置选择，不写死在领域代码。所有调用记录 provider、model、request_id、input_hash、prompt_version、schema_version、usage、latency、重试次数和输出校验结果。

18.3 供应商适配器

```
interface ModelProvider {
  generateStructured<T>(request: StructuredRequest<T>): Promise<T>;
  generateText(request: GenerateRequest): Promise<GenerateResult>;
  startBatch(requests: BatchRequest[]): Promise<BatchHandle>;
  pollBatch(handle: BatchHandle): Promise<BatchState>;
  capabilities(): ProviderCapabilities;
}
```

适配器只承诺交集能力。供应商专有功能使用 capability flag，不允许出现在核心领域对象中。

19. 自有后端 API 设计

建议主要端点：

1. POST /v1/imports：创建项目导入会话。
2. POST /v1/imports/{id}/complete：封存原始证据并启动解析。
3. GET /v1/projects/{id}/questions：查询原始问题档案。
4. POST /v1/projects/{id}/compilation-runs：启动全量或增量编译。
5. GET /v1/projects/{id}/knowledge-tree：获取知识文件树。
6. GET /v1/knowledge-documents/{id}：获取文件、区块、引用和版本。
7. POST /v1/knowledge-documents/{id}/change-proposals：生成更新提案。
8. POST /v1/change-proposals/{id}/approve：人工确认变更。
9. GET /v1/projects/{id}/timeline：查询知识演化事件。
10. POST /v1/projects/{id}/exports：导出 Markdown/ZIP。

异步端点返回 202 + job_id，支持 SSE 进度。写操作使用 Idempotency-Key，资源使用 version 做乐观锁，列表使用 cursor 分页。

20. 数据库与检索策略

PostgreSQL 保存来源结构、问答索引、知识片段、知识文件、段落、结论、引用与 Revision。对象存储保存原始导出和附件。向量用于候选召回，不作为最终归类依据。

检索顺序：权限过滤 -> 关键词召回 + 向量召回 -> reciprocal rank fusion -> 主题/范围过滤 -> 可选 rerank -> 来源窗口打包。

概念合并至少结合标准化名称、别名、上下文范围、来源重叠和模型裁决。用户应能撤销错误合并。

21. 隐私、安全与可信性

- 默认原始内容保存在用户控制的存储；发送模型前进行最小化取片。
- 导入内容始终是不可信数据，文档中的指令不能成为系统操作授权。
- API key 仅保存在服务端密钥库，日志不打印正文、附件或密钥。
- 发送前进行 PII、密钥和敏感内容检测，支持“只本地处理”标记。
- 所有 AI 生成内容保存来源类型、模型、提示与分析版本。
- 用户可以导出、撤销连接、删除来源，并触发依赖知识文件重新编译。
- OpenAI API 的 store、Files、Vector Store、后台任务和保留策略必须按账户资格与官方数据控制文档配置。

22. UI 实现顺序

UI 应在编译模型稳定后实现。第一个可用界面建议为三栏：

- 左栏：原始问答 / 知识文件树切换。
- 中栏：知识文件正文，明确区分历史归纳与 AI 补充。
- 右栏：原始引用、相关对话、变更时间线和审核操作。

必备交互包括：点击段落查看来源、按主题搜索所有问题、接受/拒绝归类、查看更新 diff、将待解决项转为新问题。视觉复杂度可以低，但来源关系必须清楚。

23. 实现版本路线

v0.1 原始项目与问答档案（第 1-2 周）

导入一个 ChatGPT 项目；保存 Conversation/Message/分支；生成问题列表并跳回原文。验收：重复导入不重复，消息定位准确率 100%。

v0.2 知识片段池（第 3-4 周）

完成 fragment 提取、主题标签、来源窗口和基础混合检索。验收：金标集片段边界 F1 >= 0.85，引用定位准确率 >= 98%。

v0.3 知识框架与文件编译（第 5-6 周）

生成项目级文件树和标准八区块 Markdown；跨对话合成并检查引用。验收：历史归纳 statement 引用覆盖率 100%，无来源断言率接近 0。

v0.4 AI 补充与人工审核（第 7-8 周）

增加独立 AI 补充区、冲突检测、接受/拒绝和发布版本。验收：补充内容不会进入 Source-derived 区域。

v0.5 增量编译与时间线（第 9-10 周）

新对话触发局部更新；支持 Append/Revise/Split/Merge/Conflict；展示段落 diff。验收：未受影响文件不重写，旧版本可恢复。

v0.6 可用 UI 与导出（第 11-12 周）

完成三栏工作台、搜索、来源回跳、审核队列和 Markdown/ZIP 导出。

v0.8 多供应商与多项目

增加 Claude/Gemini/PDF 等连接器、跨项目知识库、权限与团队协作。

v1.0 稳定知识编译平台

版本化管线、开放连接器 SDK、评测门禁、SLO、审计导出、备份与灾难恢复。

24. 测试与评测

必须建立脱敏金标项目，至少包含 20 条对话，故意让同一主题散落在不同会话，并加入冲突、重复、跑题和错误回答。

核心指标：

- Question coverage：实际问题被发现的比例。
- Fragment boundary F1：知识片段边界质量。
- Citation precision/coverage：引用是否准确、是否完整。
- Cross-conversation recall：跨会话相关片段召回率。
- Duplicate rate：知识文件中的重复比例。
- Unsupported claim rate：无来源历史结论比例。
- Framework stability：新增少量对话时目录无意义变化率。
- Human acceptance rate：用户接受归类与更新建议的比例。
- Incremental cost：每 100 条新增消息的费用与耗时。

模型、提示、Schema 或聚类策略变化都必须回放金标项目。

25. 成本与性能

确定性解析优先于模型；先用便宜任务做问题识别、片段标签和候选召回，再让高能力模型处理跨对话合成、冲突和框架调整。

缓存键包括 source hash、fragment hash、prompt/schema/model 版本。新消息只使受影响片段、概念和文件失效。全量 Batch 适合首次导入，在线增量适合日常更新。

每个项目设置 token、费用、耗时和最大框架变化预算。若一次更新建议移动大量文件，应自动进入人工审核而不是直接发布。

26. 建议仓库结构

```
GPT-SaaS/
  apps/
    web/                    # 问答与知识文件工作台
    api/                    # REST/SSE、鉴权、应用服务
  workers/
    ingestion/              # 导入与规范化
    question-index/         # 问题与回答索引
    analysis/               # 片段、主题、概念抽取
    knowledge-compiler/     # 框架、文件与增量编译
    projection/             # 时间线和发布视图
  packages/
```

```
  domain/                 # 领域对象、状态机、事件
  connectors/             # ChatGPT/Claude/Gemini/PDF
  llm-gateway/            # 模型、预算、重试、审计
  schemas/                # JSON Schema 与类型
  observability/          # trace、metrics、cost
contracts/                # OpenAPI 与事件契约
schemas/                  # 编译结果 Schema
infra/postgres/           # 数据库迁移
evals/
  fixtures/               # 脱敏对话项目
  goldens/                # 问题、片段、目录和引用金标
docs/adr/                 # 架构决策记录
output/pdf/               # 正式指导文件
tmp/pdfs/                 # 渲染检查中间文件
```

27. 主要风险与对策

风险	表现	对策
按聊天标题归类	跨主题内容被遗漏	全项目切片后再归类
总结与补充混淆	用户误以为补充来自历史	provenance_type 与独立区块
新对话导致目录震荡	文件频繁移动重命名	稳定节点 ID、变化预算、人工审核
AI 生成无来源结论	内容合理但无法验证	Source-derived 强制 citation
相同内容多处复制	知识库越来越冗余	主归属 + 引用卡片 + 重复检测
课程方法被错误泛化	特定经验变成通用规律	scope 字段与 Critique
全量重编译费用高	新增一条消息重跑全部	依赖图、哈希和增量失效
原始对话遭覆盖	无法追溯	L0 不可变，发布层单独版本化

28. 最终实施原则

这个产品最重要的不是生成“看起来很完整的总结”，而是把聊天的探索性与文件的稳定性结合起来。

一条合格的知识内容必须回答：它来自哪些对话、属于历史归纳还是 AI 补充、如何进入当前文件、用户是否确认、后来被什么新内容修订。只要这五件事可以清楚回答，系统就能真正帮助用户回顾问答、跨对话归纳、形成知识体系并持续学习。

首版的唯一主闭环应当是：导入一个学习项目 -> 查看全部问题 -> 生成跨对话知识文件树 -> 打开文件查看归纳与 AI 补充 -> 点击任一结论回到原始对话 -> 导入新对话并看到增量更新建议。

29. 参考资料（核对日期：2026-09-12）

OpenAI：Responses API（https://developers.openai.com/api/reference/resources/responses/methods/create）；Vector Store Search（https://developers.openai.com/api/reference/python/resources/vector_stores/methods/search）；Files API（https://developers.openai.com/api/reference/typescript/resources/files/methods/create）。

OpenAI：Background mode（https://developers.openai.com/api/docs/guides/background）；Batch API（https://developers.openai.com/api/reference/resources/batches）；Data controls（https://developers.openai.com/api/docs/guides/your-data）。

其他供应商：Anthropic API overview（https://platform.claude.com/docs/en/api/overview）；Gemini Batch API（https://ai.google.dev/api/batch-api）。